You can directly create components from the CLI using:

```
ng generate component
```

However, dealing with an Angular component would require you to deal with three files breaking down the same component: one that contains the TypeScript for component, one with the HTML and one with the CSS styling. Angular also has a special script to integrate your HTML and JavaScript. It allows for a special *ng script to bring Javascript into your HTML, which helps with basic functions like looping over components. Overall, while it makes understanding a particular block of code easy, learning Angular to build an application from scratch is a very strenuous task. Angular is mostly used for enterprise applications.

EXHIBIT C: VUE

Vue is a lot like Angular, but it is structured in a way that benefits individual developers as well. It has official packages for routing and state-management with a large ecosystem of third-party packages. Vue has an extremely powerful CLI. For example, typing "vue ui" will render a whole application on your browser which will walk you through all of the different dependencies and features to add while generating the initial app. However, you cannot create components via this CLI.

The project structure for a Vue app is more minimal than Angular. However, there is a different plugin system to add properties like routing and state-management as needed. Code in Vue is organised in three parts in a .vue file: a template, the script, and the style. Every component is a simple Javascript object. State-management is done internally using keywords such as "data" (to initialise a component) and "methods" (to add functionality when an event is triggered). Vue makes understanding frameworks extremely simple as it adds the same functionalities as React or Angular but on a plain Javascript object instead of a React class or Angular component. Basically, Vue is a simpler option than Angular for independent developers.

EXHIBIT D: SVELTE

Like React, Svelte is designed as a basic component library. Other features such as routing are sourced by the community itself, and can be integrated into your Svelte app using third-party packages. However, since Svelte does not have a big community like React, there are fewer external packages to choose from.

Now, a virtual DOM is a system where an ideal, or "virtual", representation of a UI is kept in memory and synced with the "real" DOM by a library. This VDOM system exists in all of the above frameworks. However, Svelte does not keep a virtual DOM. It acts as a JavaScript compiler, to turn your code into JavaScript. On initialising a Vue project, you will notice a rollup.config.js file which is used to bundle up your code. This is basically referred to as a module bundler. Svelte is more hands-on in this area, and the developer might have to learn a little about module bundlers in order to execute some code. Other frameworks usually abstract this part of the functioning from the developer.

The file structure of a .svelte file is the same as a .vue file with three parts for each component. Svelte provides a different syntax for integration of HTML and Javascript. For example, it has the "#each" keyword to loop over components. Because of the number of files and its non-DOM nature, Svelte is extremely lightweight. For an application that does not require heavy animation or very detailed styling, Svelte is great. While already knowing a React makes learning Angular easier, learnings from Svelte is not easily transferable to other frameworks.

EXHIBIT E: ALPINE

Alpine is a tiny library of around 4 kB that allows you to extend your existing HTML with reactive data to contain many features that are supplied by other heavy frameworks. Alpine takes the focus away from optimising Javascript to perfecting your HTML. If you have used Tailwind for CSS, Alpine is the JavaScript equivalent. Currently, it serves as a popular replacement for the now-redundant jQuery.

You can import the Alpine script as a link in your HTML script tag. It brings your Javascript into HTML, right from defining your object to performing functionalities on it. Any object or component declared in an HTML tag using the "x-data" keyword can be used by child attributes. Additionally, Alpine has a mechanism called "Alpine.store", which allows you to store and share data between multiple components in the UI. Alpine is the absolute best option when you want to add a little bit of interactivity to an already-existing HTML page. However, it is not intended to be a complete framework that can replace something like Angular or React.

EXHIBIT F: MITHRIL

Mithril is extremely lightweight and uses a virtual DOM like React or Vue, but the overall developer experience differs from these frameworks. It starts just like Alpine: you create an index.html file and add the Mithril script to the tag. However, instead of creating component classes or using a CLI, in Mithril you can create a component using simple JavaScript. You can add data and functionality in terms of methods to the component object itself. To link the JavaScript and HTML, you can use the "m(..)" keyword where you pass the required HTML component like "ul" or "form" along with functions and data. Since the framework is such a unique mix of Alpine, React and JSX, it takes a lot of time to learn and get used to. Just like Svelte, learning this will not give you an idea of how other frameworks function.

For someone who doesn't like to work with HTML, Mithril is perfect as JavaScript does most of the heavy lifting. However, most devs in today's world are used to an HTML-first development cycle. Hence, working with something like Mithril feels very awkward at first. But if you are someone who is already familiar with React or Angular and loves learning, Mithril might be an interesting challenge to occupy your mind for sometime.

As mentioned at the start, there is no best framework. Each has its unique implementation in specific scenarios. In the end, you should choose the one that you think will work best for you. Who knows, by the time you finish reading this article, these existing frameworks might have modified themselves to fit other requirements. d